



From Syntax to Solution: Basic Applications of Python to Data Science

DLSU Data Science Society Python Workshop

Engr. Jaydee N. Lucero

Senior Structural Engineer I, Abinales Associates Engineers + Consultants

May 27, 2026

Hi! I'm Jaydee.

- * *Senior Structural Engineer I* (2021-present), Abinales Associates Engineers + Consultants
- * *Part-time Review Instructor* (2019-present), Review Innovations
- * *Associate Member* (2022-present), Association of Structural Engineers of the Philippines, Inc.
- * *MS Civil Engineering (Structural Engineering) student* (2024-present), University of the Philippines Diliman
- * Pythonista since August 2024



Outline

1. Writing in Python: why?
2. Writing Python syntax
3. Writing code Pythonically
4. Writing faster code
5. Writing functions that take in functions
6. Writing code using the "Big 3" libraries
7. Writing the next steps

QR Codes

Live codes for this talk



Python 3.14 cheat sheet



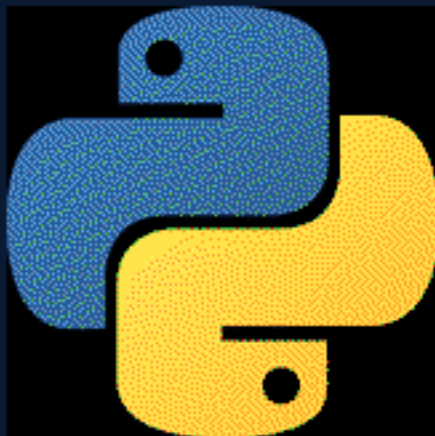
Part 1

Writing in Python: Why?

Why Python?

[Python website](#)

- * Python is powerful.
- * Python is fast.
- * Python plays well with others.
- * Python runs everywhere.
- * Python is friendly.
- * Python is easy to learn.
- * Python is open[-source].



Why Python?

Python website

- * Python is powerful.
- * Python is fast.
- * Python plays well with others.
- * Python runs everywhere.
- * Python is friendly.
- * Python is easy to learn.
- * Python is open[-source].

A C program to determine the maximum value in an array

```
1 #include <stdio.h>
2
3 int main(void) {
4     int arr[] = {3, 1, 6, 4};
5     int n = 4;
6     int max = arr[0];
7     int i;
8
9     for(i = 1; i < n; i++)
10         if(arr[i] > max)
11             max = arr[i];
12
13     printf("Max = %d\n", max);
14     return 0;
15 }
```

Equivalent Python program

```
1 arr = [3, 1, 6, 4]
2 print(max(arr))
```

Why Python?

Python website

- * Python is powerful.
- * Python is fast.
- * Python plays well with others.
- * Python runs everywhere.
- * Python is friendly.
- * Python is easy to learn.
- * Python is open[-source].

TIOBE Index, May 2026

Rank	Language	Rating
1	Python	19.98%
2	C	11.55%
3	Java	7.94%
4	C++	7.92%
5	C#	5.41%
6	Javascript	3.08%
7	VB	2.90%
8	R	1.77%
9	SQL	1.57%

Why Python?

Python is powerful.

Some of the Python libraries that we need in data science.

Library	Function
numpy	arrays, fast math, linear algebra
scipy	numerical math, stats, regression
scikit-learn	machine learning applications
pandas	data extraction, cleaning, analysis
matplotlib	data visualization

Part 2

Writing Python syntax

Variables and data types

We can declare a variable in the same way we do in math.

```
1 | age = 29
```

Variables can hold any type of data possible, some of which are enumerated below.

```
1 name          = "Airis"           # string
2 gender        = 'F'               # character
3 age           = 21                 # integer
4 money         = 125.36             # float (decimal number)
5 is_single     = True               # boolean
6 has_dogs      = None               # the None keyword
7 siblings     = ["Eris", "Erin", "Erina"] # list
8 siblings_age  = (36, 25, 18)       # tuple
9 siblings_gender = {'F', 'F', 'F'}  # set
10 parents      = {                  # dictionary
11     "mother"      : "Airin",
12     "mother_age"   : 50,
13     "father"      : "Erik",
14     "father_age"   : None
15 }
```

Program output

Texts and variable values can be displayed using the `print` function.

```
1 | print("Hello world!")    # displays "Hello world!"
2 | print(money)             # displays the value of the variable money
```

They can even be combined together using either

* the `.format` function

```
1 | print("Hello world! I'm {0} and my money is Php{1}.".format(name, money))
```

* or, f-strings

```
1 | print(f"Hello world! I'm {name} and my money is Php{money}.")
```

Mathematics and Library imports

You can do some simple math calculations in Python.

```
1 print(3 + 5 - 3 * 5)    # addition, subtraction, multiplication
2 print(3 / 5)            # float division      (3/5 = 0.6)
3 print(3 // 5)           # integer division    (3/5 = 0 remainder 3, so 0)
4 print(3 % 5)            # remainder          (3/5 = 0 remainder 3, so 3)
5 print(3 ** 5)           # exponents          (3^5 = 243)
6 print(max(3, 5))        # maximum value
7 print(min(3, 5))        # minimum value
```

More advanced math functions require the math library. Import and use using

```
1 import math
2 print(math.sqrt(25))    # square root
```

We can import only select functions from a specific library.

```
1 from math import sqrt   # import just the square root function
2 print(sqrt(25))
```

Or import all functions in that library using

```
1 from math import *      # import everything
2 print(sqrt(25))
```

Control statements and loops

Check multiple conditions using if...elif...else.

```
1 age = 29
2
3 if age < 18 :
4     print("A minor.")
5 elif 18 <= age < 65 :
6     print("An adult.")
7 else :
8     print("A senior citizen.") # An adult.
```

Do repetitive operations using loops.

```
1 # Calculate the sum of the first 100
2 # positive integers.
3 sum = 0
4
5 for num in range(1, 101) :
6     sum += num
7
8 print(sum)    # 5050
```

We can also combine conditions and repetitive operations using while.

```
1 # Another way to calculate the sum of the
2 # first 100 positive integers.
3 sum = 0
4
5 current_num = 1
6 while current_num <= 100 :
7     sum += current_num
8     current_num += 1
9 print(sum)    # 5050
```

or

```
1 sum = 0
2
3 current_num = 1
4 while True :
5     sum += current_num
6     if current_num > 100 :
7         break
8 print(sum)    # 5050
```

Lists, tuples, dictionaries

Access the elements in lists and tuples using

```
1 # index 0, 1, 2, 3
2 # index -4, -3, -2, -1
3 a = [1, 2, 3, 4]
4 print(a[1])           # 2
5 print(a[-2])          # 3
```

and for dictionaries,

```
1 person = {
2     "name" : "James",
3     "age"  : 41
4 }
5 print(person["name"]) # James
```

You can also access multiple elements at once using the `:` symbol.

```
1 # from index 1 (included) to 3 (excluded)
2 print(a[1:3])           # [2, 3]
```

Modify elements in lists using

```
1 a[1] = 100
2 print(a)           # [1, 100, 3, 4]
```

and in dictionaries using

```
1 person["age"] += 1
2 print(person["age"]) # 42
```

You can't do that on tuples though since they're immutable.

```
1 b = (1, 2, 3, 4, 5)
2 b[2] = 100
3 # TypeError: 'tuple' object does not
4 # support item assignment
```

Add new elements to lists using

```
1 a.append(200)
2 print(a)           # [1, 100, 3, 4, 200]
```

Part 3

Write code Pythonically

Writing code Pythonically

Example 1 Find the sum of the first 100 positive perfect square numbers.

Manual computation

$$1^2 + 2^2 + \dots + n^2 = n(n + 1)(2n + 1)/6$$

$$1^2 + 2^2 + \dots + 100^2 = 100(100 + 1)[2(100) + 1]/6 = 338350$$

Solution 1 using for loops

```
1 total = 0
2 for n in range(1, 100 + 1) :
3     total += n**2
4 print(total)
```

Solution 2

```
1 print(sum([n**2 for n in range(1, 100 + 1)]))
```

List comprehensions

`[(1) for (2) in (3) if (4)]` generates a list of items generated by part (1) for each item (2) in the iterable (3) based on conditions set by (4).

Writing code Pythonically

Example 2 Suppose that we have the following record of transactions in US Dollars. Notice that some transactions are corrupted (the amount is either None or negative value). We want to determine the total amount of all valid transactions only, converted to Euros using the conversion 1 US Dollar = 0.85 Euro.

```
1 transactions_usd = [10.50, None, -5.00, 25.00, 100.00, None, 12.50]
2 usd_to_euro = 0.85
```

Solution

```
1 transactions_usd = [10.50, None, -5.00, 25.00, 100.00, None, 12.50]
2 usd_to_euro = 0.85
3
4 total_euros = sum([amount * usd_to_euro for amount in transactions_usd
5                   if amount is not None and amount > 0])
6 print(total_euros)
```

Part 4

Writing faster code

Writing faster code

Some ways to make code faster

- * Use `numpy` arrays instead of lists.
- * Use raw iterator objects instead of lists when using all its contents.
- * Use dictionaries instead of lists.
- * Use sets instead of lists.

Containment time complexities

Data struct	Average	Worst case
List/Tuple	$O(n)$	$O(n)$
Dictionary	$O(n)$	$O(n)$
Set	$O(1)$	$O(n)$

Iteration time complexities

Data struct	Average	Worst case
List/Tuple	$O(n)$	$O(n)$
Dictionary	$O(n)$	$O(n)$
Set	$O(n)$	$O(n)$

Writing faster code

Creating lists

```
1 import time
2
3 time_start = time.time()
4 numbers = [num for num in range(1, int(1e8) + 1)]
5 time_end = time.time()
6
7 print(f"time = {time_end - time_start:.4f} seconds.")    # 1.6669 seconds
```

Creating numpy arrays

```
1 import time
2 import numpy as np
3
4 time_start = time.time()
5 numbers = np.arange(1, int(1e8) + 1)
6 time_end = time.time()
7
8 print(f"time = {time_end - time_start:.4f} seconds.")    # 0.0978 seconds
```

About 17 times faster!

Writing faster code

Finding elements in lists

```
1 import time
2 numbers = [num for num in range(1, int(1e8) + 1)]
3
4 time_start = time.time()
5 number_found = int(1e8) in numbers
6 time_end = time.time()
7 print(f"time = {time_end - time_start:.4f} seconds.")    # 0.4242 seconds
```

Finding elements in sets

```
1 import time
2 numbers = set([num for num in range(1, int(1e8) + 1)])
3
4 time_start = time.time()
5 number_found = int(1e8) in numbers
6 time_end = time.time()
7 print(f"time = {time_end - time_start:.6f} seconds.")    # 0.000005 seconds
```

About 80,000 times faster!

Part 5

Writing functions that take in
functions

Functions as objects

Functions as objects

Function *names* can be inputted as arguments in a function.

Example 3 Solve the equation $e^x = x + 5$ using Newton-Raphson method.

Solution

```
1 import math
2
3 # The equation, expressed as left - right. This should be equal to zero.
4 eqn = lambda x : math.exp(x) - x - 5
5
6 # Define a function that takes an equation (a function) as an argument.
7 def newton_raphson(func, x0=0, tol=1e-6) :
8     xn = x0      # guess value
9     h = 1e-6     # marching step for numerical differentiation
10    # Check if the equation is close to zero.
11    while abs(func(xn)) >= tol :
12        d_func = (func(xn + h) - func(xn - h))/(2*h)    # derivative
13        xn -= func(xn)/d_func                            # new guess value
14    return xn      # root found
15
16 print(newton_raphson(eqn, x0=1))          # 1.936847407229323
```


Functions as objects

Applications to pandas

The pandas apply function

`[pandas Series].apply([function])` takes a *pandas* Series, feeds its contents one by one into a function, then returns a new *pandas* Series.

Example 4 UP Diliman implements the following honorific scholarships based on the general weighted average (GWA) of a student for a given semester.

Item	Range
University Scholar (US)	$GWA \leq 1.45$
College Scholar (CS)	$1.45 < GWA \leq 1.75$
No scholarship	$GWA > 1.75$

Given a *pandas* DataFrame of students and their corresponding GWA for a specific semester, output the honorific scholarship of each student.

Functions as objects

Applications to pandas

Solution

```
1 import pandas as pd
2
3 data = {
4     "Students" : ["Jinx", "Lux", "Ahri", "Annie", "Ezreal"],
5     "GWA"       : [1.1356, 1.2578, 1.6703, 1.8935, 2.2410]
6 }
7 data = pd.DataFrame(data)
8
9 def scholarship(gwa) :
10     if gwa <= 1.45 :
11         return "University Scholar (US)"
12     elif gwa <= 1.75 :
13         return "College Scholar (CS)"
14     else :
15         return "No scholarship"
16
17 data["Scholarship"] = data["GWA"].apply(scholarship)
18 print(data)
```

Part 6

Write code using the "Big 3"
libraries

The numpy library

Example 5 Given the system of linear equations

$$5x + 3y + 2z = 15$$

$$2x - 6y - 7z = -22$$

$$4x - 8y + 3z = 9$$

find the value of $x^2 + y^2 + z^2$.

Solution

```
1 import numpy as np
2
3 A = [[5, 3, 2],      # coefficients matrix
4       [2, -6, -7],
5       [4, -8, 3]]
6 B = [15, -22, 9]     # constants vector
```

Using matrix inverse method,

```
1 X = np.linalg.inv(A) @ B
```

or, using *numpy*'s own solver,

```
1 X = np.linalg.solve(A, B)
```

Either way, we have

```
1 result = np.sum(X**2)
2 print(result)
```

The numpy Library

Example 6 The Civil Engineering Licensure Examination has three subjects with codes MSTC, HGE and PSAD. A future engineer is required to take all these three subjects. His/Her overall rating is then calculated according to the formula

$$\text{rating} = 0.35 \times \text{MSTC} + 0.30 \times \text{HGE} + 0.35 \times \text{PSAD}$$

He/She is said to have passed the exam if the following conditions are satisfied:

- * $\text{rating} \geq 70$
- * $\text{MSTC}, \text{HGE}, \text{PSAD} \geq 50$

Given the results from ten students as shown below, determine whether each student has passed the exam.

```
1 # student      1    2    3    4    5    6    7    8    9   10
2 scores_MSTC = [96, 92, 89, 83, 72, 66, 59, 51, 42, 38]
3 scores_HGE  = [88, 85, 84, 80, 83, 68, 52, 63, 30, 28]
4 scores_PSAD = [90, 49, 83, 75, 86, 47, 56, 62, 96, 58]
```

The numpy library

Solution 1 using lists

```
1 scores_MSTC = [96, 92, 89, 83, 72, 66, 59, 51, 42, 38]
2 scores_HGE = [88, 85, 84, 80, 83, 68, 52, 63, 30, 28]
3 scores_PSAD = [90, 49, 83, 75, 86, 47, 56, 62, 96, 58]
4
5 rating = [0.35*mstc + 0.30*hge + 0.35*psad \
6           for mstc, hge, psad in zip(scores_MSTC, scores_HGE, scores_PSAD)]
7 is_passed = [rating >= 70.0 and mstc >= 50.0 and hge >= 50.0 and psad >= 50.0 \
8              for rating, mstc, hge, psad in \
9                  zip(rating, scores_MSTC, scores_HGE, scores_PSAD)]
10
11 print(rating)
12 print(is_passed)
```

NOTE The *zip* function aggregates elements from multiple iterables (like lists, tuples, or strings) into a single iterator of tuples. Each tuple contains elements that share the same index in their original sequences.

The numpy library

Solution 2 using *numpy* arrays

```
1 scores_MSTC = [96, 92, 89, 83, 72, 66, 59, 51, 42, 38]
2 scores_HGE = [88, 85, 84, 80, 83, 68, 52, 63, 30, 28]
3 scores_PSAD = [90, 49, 83, 75, 86, 47, 56, 62, 96, 58]
4
5 import numpy as np
6
7 scores_MSTC = np.array(scores_MSTC, dtype='float')
8 scores_HGE = np.array(scores_HGE, dtype='float')
9 scores_PSAD = np.array(scores_PSAD, dtype='float')
10
11 rating = 0.35*scores_MSTC + 0.30*scores_HGE + 0.35*scores_PSAD
12 is_passed = (rating >= 70.0) & (scores_MSTC >= 50.0) & (scores_HGE >= 50.0) \
13             & (scores_PSAD >= 50.0)
14
15 print(rating)
16 print(is_passed)
```

The pandas library

Example 7 Given the population of the Philippines from 1840 to 2024 based on national census, answer the following questions.

1. The population growth of a certain country is said to be *sustainable* if the annual growth rate is between 0.5% and 1.2%. List out the years in which the population growth of the Philippines was sustainable.
2. Based from the provided population data, calculate the average annual growth rate in percent for each listed year.
3. Based from the provided annual growth rate, calculate the maximum and minimum growth rate, and the respective years in which they occurred.
4. Create the following: (a) a bar graph of population for each year, (b) multiple line graphs in a single figure, covering population, population increase and provided and calculated annual growth rate.

The pandas library

Solution #1

```
1 | import pandas as pd
```

pandas can import Excel and CSV files.

```
1 | ph_population = pd.read_excel("2025_T1_4.xlsx")
```

Imported data can sometimes contain invalid data like NaN (not a number) or None values. `dropna()` can be used to remove these values.

```
1 | ph_population = ph_population.dropna()
```

Like lists or *numpy* arrays, we can specify the rows that we will get as data.

```
1 | ph_population = ph_population[1:]    # Exclude the first row (i.e. index 0).
```

`.columns` can be used to specify the column names.

```
1 | ph_population.columns = ["Year", "Population", "Annual Growth Rate (%)", "Source"]
```

The pandas library

Solution #1

We can filter the data by specifying conditions within square brackets.

```
1 sustainable = ph_population[ph_population["Annual Growth Rate (%)"] >= 0.5]
2 sustainable = ph_population[ph_population["Annual Growth Rate (%)"] <= 1.2]
```

We can also specify which columns will be considered.

```
1 print(sustainable[["Year", "Annual Growth Rate (%)"]])
```

Year	Annual Growth Rate (%)
1870	0.78
1887	0.72
1896	0.5
2024	0.8

The pandas library

Solution #2

`.diff()` calculates the difference between consecutive values.

```
1 ph_population["Annual Increase"] = ph_population["Population"].diff() / \
2                               ph_population["Year"].diff()
```

`.shift()` shifts the values in a column by a specified number of periods.

```
1 ph_population["Calculated Rate (%)"] = ph_population["Annual Increase"] / \
2                               ph_population["Population"].shift(1) * 100
```

Show only the last two rows.

```
1 print(ph_population[["Year", "Population", "Annual Increase",
2                               "Calculated Rate (%)"]][-2:])
```

Year	Population	Annual Increase	Calculated Rate (%)
2020	109033245	1610788.4	1.595167
2024	112729484	924059.75	0.847503

The pandas library

Solution #3

Some statistical measures that can be computed using *pandas* are `max()`, `min()`, `mean()`, `median()`, `std()`, `count()` and `sum()`.

```
1 # Maximum and Minimum annual growth rates.
2 print(ph_population["Annual Growth Rate (%)"].max())    # 3.08
3 print(ph_population["Annual Growth Rate (%)"].min())    # 0.5
```

`.idxmax()` and `.idxmin()` returns the index numbers of the maximum and minimum values, respectively.

Using that index, we can retrieve the corresponding row using `.loc[]`.

```
1 # Year with which they respectively occurred.
2 print(ph_population.loc[ph_population["Annual Growth Rate (%)"].idxmax(), "Year"]) # 1970
3 print(ph_population.loc[ph_population["Annual Growth Rate (%)"].idxmin(), "Year"]) # 1896
```

The matplotlib library

Solution #4 (a)

`.to_numpy()` converts a pandas Series to a *numpy* array.

```
1 import matplotlib.pyplot as plt
2
3 year = ph_population["Year"].to_numpy(dtype='float')
4 population = ph_population["Population"].to_numpy(dtype='float')
```

From this, we can plot the data using *matplotlib*.

```
1 # A vertical bar plot
2 fig, ax = plt.subplots()
3 ax.bar([str(y) for y in year], population) # Convert years from numbers to strings
4 ax.tick_params("x", rotation=90)          # Rotate x-axis labels by 90 degrees
5 ax.set_xlabel(ph_population.columns[0])    # Set x-axis label
6 ax.set_ylabel(ph_population.columns[1])    # Set y-axis label
7 ax.set_title("Philippine Population (PSA Census Data)") # Set plot title
```

The matplotlib library

Solution #4 (b)

Several plots can be placed in a single figure using `plt.subplots()`.

```
1 fig, ax = plt.subplots(1, 4)                # Set up 4 subplots in a row
2 fig.set_size_inches(24, 6)                  # Set the overall figure size
3
4 for i, cur_col in enumerate((1, 2, 4, 5)) : # Loop through the columns we want to plot
5     # Plot the data
6     ax[i].plot(year, ph_population[ph_population.columns[cur_col]], 'ro-', linewidth=2)
7
8     # Set the axis labels
9     ax[i].set_xlabel(ph_population.columns[0])
10    ax[i].set_ylabel(ph_population.columns[cur_col])
11
12    # Set the title
13    ax[i].set_title(ph_population.columns[cur_col])
```

The scikit-learn library

Example 8 Given an Excel file of 200 rebound hammer test results reported from different parts of a single building, construct a best-fit line that can be used to estimate the compressive strength (in MPa) of a part of the same building given the obtained rebound number. Comment on the results.

Solution

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import r2_score
```

As usual, read the Excel file and extract the data as *pandas* Series.

```
1 df = pd.read_excel("rebound_hammer_data.xlsx")
2 rebound_num = df[["Rebound Number"]]
3 strength = df["Concrete Strength (MPa)"]
```

Note: *scikit-learn* requires two-dimensional *x*-data. Hence, the use of double brackets `df[["..."]]` in line 2 above.

The scikit-learn library

Create a LinearRegression model, and fit it to the given data.

```
1 model = LinearRegression()  
2 model.fit(rebound_num, strength)
```

Calculate the properties of the best-fit line.

```
1 slope      = model.coef_[0]  
2 intercept  = model.intercept_  
3 r2         = r2_score(strength, model.predict(rebound_num))
```

Construct the regression line for plotting.

```
1 x_line = np.linspace(rebound_num.min(), rebound_num.max(), 300)  
2 y_line = model.predict(x_line)
```

Finally, plot the data points and the regression line.

```
1 fig, ax = plt.subplots()  
2 ax.scatter(rebound_num, strength, label="Data points")  
3 ax.plot(x_line, y_line, color="tomato", linewidth=2, label="Regression line")  
4 ax.set_xlabel("Rebound Number")  
5 ax.set_ylabel("Concrete Strength (MPa)")  
6 ax.legend()
```


Part 7

Writing the next steps

Next steps

1. Create your Github account.
2. Read the library documentation **a lot**.
3. Stop coding examples. Start building your own projects.
4. Feel free to make mistakes.
5. Feel free to ask questions.
6. When using AI, **always** verify the results.
7. Find your desired specialization (e.g. actuarial science, finance, AI/ML, software and game development, etc).
8. Join a community (e.g. Reddit, Discord, community groups, etc).
9. Continue learning.

Closing

This presentation was created using a heavily modified version of [Pyxel slides](#), which I learned at PythonAsia 2026 from the presentation of [Takayuki Shimizukawa](#). Special thanks to Shimizukawa-sensei for creating such an amazing tool!

Contact me

- * Email: jaydee.lucero@gmail.com
- * Facebook: facebook.com/jaydee.lucero
- * Linkedin:
linkedin.com/in/jaydee-lucero-977070200
- * Github: github.com/arisa-chan
- * Website: enqr.jaydee.com

PDF and source code of presentation



Thank you very much!
Do you have any questions?